

T.C.
YILDIZ TECHNICAL UNIVERSITY
FACULTY OF MECHANICAL ENGINEERING
DEPARTMENT OF INDUSTRIAL ENGINEERING
YOLO INSTANCE SEGMENTATION ROBUSTNESS AGAINST DIFFERENT
IMAGE COMPRESSIONS

Abdullah Kefah Taha Issa

Advisor
Prof. Dr. Alev Taskin

UNDERGRADUATE GRADUATION THESIS

January, 2026

TABLE OF CONTENTS

LIST OF SYMBOLS	3
LIST OF ABBREVIATIONS	4
ABSTRACT	7
INTRODUCTION	1
LITERATURE REVIEW	5
Corruption Robustness Benchmarks	5
Effects of Compression on CV models	6
Practical Application of Image Compression	8
Shortcomings and Contributions	8
METHODOLOGY	15
Research Method	15
Problem Definition	15
Dataset and Data Collection	15
Methods, Techniques, and Tools Used	15
Analysis Plan and Implementation Steps	18
Assumptions and Limitations	20
RESULTS AND ANALYSIS	21
Performance by File Size	21
Efficiency Ratio Analysis	23
Weighted Compression Score	24
Marginal Efficiency	25
Threshold-Based Observations	27
CONCLUSION	28
Bibliography	29

LIST OF SYMBOLS

CS	Compression Score
ER	Efficiency Ratio
mAP	mean Average Precision
ME	Marginal Efficiency
WCS	Weighted Compression Score

LIST OF ABBREVIATIONS

AI	:Artificial Intelligence
CV	:Computer Vision
CNN	:Convolutional Neural Networks
ML	:Machine Learning
SSIM	:Structural Similarity Index Measure
YOLO	:You Only Look Once

LIST OF FIGURES

Figure 1 Visualization of how YOLO detection works	16
Figure 2 Implementation steps	19
Figure 3 Bar chart of Average file size of each format and its respective performance (Mask mAP@50 95)	21
Figure 4 mAP Performance by File Size - Bar Chart (Ordered by size)	22
Figure 5 mAP Performance by File Size - Bar Chart (Ordered by mAP)	22
Figure 6 mAP Performance by File Size - Line Graph	23
Figure 7 Efficiency Ratio vs Quality Level by Format	24
Figure 8 Marginal Efficiency across quality transitions by format. The dashed threshold line represents $ME = \lambda$, with $\lambda = 1$ as the default shown.	27

LIST OF TABLES

Table 1 Literature Summary	10
Table 2 Top configurations ranked by Weighted Compression Score at $\lambda=0.5$ (Storage Priority)	24
Table 3 Top configurations ranked by Weighted Compression Score at $\lambda=2$ (Balanced Priority)	25
Table 4 Top configurations ranked by Weighted Compression Score at $\lambda=5$ (Accuracy Priority)	25
Table 5 Marginal Efficiency across quality transitions for HEIC (top), JPEG (middle), and WEBP (bottom)	25
Table 6 Configurations achieving highest size reduction at different minimum mAP thresholds	27

ABSTRACT

In computer vision systems, lossy images are used to reduce storage and transmission costs, with the de facto format being JPEG. Nevertheless, with newer image formats being introduced, namely WebP and HEIC, a comparison and evaluation between formats and compressions is necessary.

This study benchmarks the impact of JPEG, HEIC, and WebP on the YOLO11n instance segmentation model, focusing on the trade-off between storage efficiency and model performance retention. The model is trained on PNG images and the test dataset is converted to different compression levels of each format. Performance is evaluated using a framework of metrics: size reduction, mAP drop, Compression Score, Weighted Compression Score, Efficiency Ratio, and Marginal Efficiency, each capturing a different aspect of the storage-performance tradeoff.

The results show that HEIC and WebP provide a more favorable storage-performance balance than JPEG across most quality settings. HEIC is the only format with multiple efficient compression operating points, while JPEG offers at most one before performance degrades sharply. These findings provide empirical evidence for image format selection in storage-constrained segmentation pipelines.

Keywords: image compression, instance segmentation, YOLO, storage efficiency, compression tradeoff

INTRODUCTION

In the last decade, there has been a surge in Machine Learning (ML) and Artificial Intelligence (AI), specifically in Computer Vision (CV). Computer Vision is the field of AI that focuses on having computers derive information from digital images. CV experienced a surge in performance with the adoption of Convolutional Neural Networks (CNN), which introduced a new paradigm for image classification. AlexNet, for one, was an architecture which used Deep CNN to achieve breakthrough results and win the ImageNet challenge (Krizhevsky et al., 2012). Then, it carried on with one-stage and two-stage object detection and segmentation with R-CNN (Girshick et al., 2014), Faster-RCNN (Ren et al., 2016), and YOLO (Redmon et al., 2016).

YOLO (You Only Look Once) is a one-stage object detection algorithm based on the paper by Redmon et al. (2016). While two-stage methods such as Mask R-CNN (He et al., 2018) generates region proposals first then undergoes the prediction, YOLO proposes directly processing the image in a single step, directly predicting the object classes and bounding boxes. YOLO started as an object detection algorithm, and has evolved ever since to also include segmentation, pose estimation, tracking, and classification (Ultralytics, 2026).

Ever since, there have been various applications of CV in human lives. In the medical field, classification and detection tasks are used in diagnostics to classify skin lesions as cancerous or not (Esteva et al., 2017). Autonomous driving was made possible with the aid of segmentation and detection, enabling the device to sense real-time pedestrians, cars, and traffic lights (Bojarski et al., 2016) . In addition, there is the task of Face Recognition which is now a standard for biometric verification. (Schroff et al., 2015)

When it comes to Segmentation tasks, a pivotal part of training and predictions is the quality of the input image, which relies heavily on the image format and compression. Image compression

is the technique that “reduce the amount of memory used by reducing the number of bits without losing the important data” (Jalilian et al., 2022). Each image format has a different method used for compression, in which file size is reduced by encoding the data in a more efficient manner. Therefore, the selection is crucial to get the optimal storage-performance trade-off, where it balances between the compression of the file size and how much original data is retained. The quality would directly affect the models performance, either positively or negatively, while the size is responsible for the storage cost.

There is a lack of a clear benchmark for the performance of YOLO Instance Segmentation models when it comes to different image formats. The absence of this type of benchmark creates a narrow space of options when it comes to selecting an image format that would serve best to a specifically tailored use case where the balancing between performance and size is necessary, forcing users to choose between options that might not be optimal.

The application of segmentation tasks is somewhat in a state of preference for JPEG over any other image format, which is apparent in the widespread hardware and software support (Reich et al., 2024). Manufacturers and developers alike are creating dedicated hardware and pipelines especially made for JPEG formats to make it easier and faster to decode and get results. For example, Nvidia provides the nvJPEG library, which is a high-performance GPU-accelerated library for decoding, encoding and transcoding JPEG format images (NVIDIA, 2026). This state is called a technological lock-in, where the main notion is “as economic and cultural advantages accrue to existing incumbent technologies, barriers are created to the adoption of potentially superior or at least as valuable alternatives.” (Foxon, 2013) . This limits competition with other image formats which could be utilized to optimize performance or storage size.

This problem introduces a couple of issues:

- **Constraining Edge Devices:** This problem affects the deployment of segmentation models on storage constrained settings, i.e. edge devices. Edge devices need every optimization it can

with storage and bandwidth, and since there is no clear benchmark on how different formats affect performance, sticking to the safe side -which is the reliance on JPEG by convention- is chosen.

- **High Storage Costs:** Nowadays, There are large amounts of data processed and inferred daily, reaching to 40 TB of data in the case of autonomous vehicles case (Faykus et al., 2022), requiring significant storage which companies pay for monthly. The problem hinders the most basic form of optimizing cost by relying on JPEG by convention only.

In this study, the aim is to produce a performance benchmark which gives a storage efficiency and inference accuracy indicator for three lossy image formats (JPEG, HEIC, WEBP) on the YOLO instance segmentation model. The objective of this study is observing how different image format provides different storage performance trade-off. This benchmark would act as supporting empirical evidence for image format selection in segmentation pipelines.

For this study, a YOLO instance segmentation model will be trained using the CityScapes dataset (Cordts et al., 2016). The training will be done in the dataset's original format which is PNG, a lossless format, and that will act as a baseline model. Afterwards, the dataset's testing split will be converted into multiple lossy formats and compressions, then inference and recording of the results will be conducted. At the end, a comparison will be conducted between the segmentation performances and sizes and compression rates that the different formats produced.

One of the limitations of this study is that it only includes a YOLO-based architecture and 3 image formats, alternative architectures such as Mask R-CNN (He et al., 2018) and image formats such as JPEG 2000 (Wikipedia contributors, 2026) will not be evaluated. The study also focuses on Instance segmentation, a specific CV task, hence excluding object detection, pose estimation, semantic segmentation, and other computer vision tasks. Finally, While latency, bandwidth, and hardware and software specific aspects are important in this topic, they are

beyond the scope of this study.

Hendrycks & Dietterich (2019), Michaelis et al. (2020), Kamann & Rother (2020) all introduced robustness benchmark datasets to measure the robustness of different models and tasks to various types of image corruption. Each tackling a different task, they agreed that the overall trend across different models and tasks is that digital corruption negatively affected the performance. In a more specific domain, Poyser et al. (2020) and Reich et al. (2024) were the most extensive papers tackling the topic of Image Compression impact on performance, in which they analyzed the impact of JPEG compression on different CV models and tasks.

At the end of this study, the contributions towards the literature would be:

- a Benchmark of YOLO segmentation model on different Compression levels, showing how each compression retains performance
- Evaluation of HEIC, JPEG, and WebP compressions as solutions and alternatives for a storage-performance balance

LITERATURE REVIEW

In the beginning of this section, the benchmark datasets proposed to measure Corruption Robustness of a model relative to the distribution shifts produced by Image Corruption with different types (Blur, Noise, Weather, Digital) are covered. Following that, Analysis of the impacts of JPEG compression on the performance and accuracy, while providing mitigation strategies is viewed. Finally, a real-life case where performance storage trade-off was critical and pivotal in the solution provided is discussed.

There are two definitions that should be introduced before heading to the papers' review, Image Corruption and Corruption Robustness. Image Corruption is defined as “visible distortions applied to images, causing a shift in data distribution from that of the original training data.” (Wang et al., 2024), while Corruption Robustness refers to “the ability of a computer vision model to withstand image corruptions that are unexpected and may occur in the test samples without significant degradation of performance.” (Wang et al., 2024).

Corruption Robustness Benchmarks

Hendrycks & Dietterich (2019) produced a “rigorous benchmark for Image Classifiers robustness”. By applying 15 diverse corruption types and a total set of 75 common visual corruptions to the ImageNet dataset, they established ImageNet-C. These corruption types are based on four categories: noise, blur, weather, and digital. Testing it on a variety of models and classifiers, they have found, despite absolute increasing performance from AlexNet to ResNet, that the architectural advancements in these classifiers have not any major change in the relative corruption robustness.

Following in their footsteps, Michaelis et al. (2020) proposed a robust detection benchmark for

object detection models, producing Pascal-C, Coco-C and Cityscapes-C. They implemented the same corruption types presented in Hendrycks & Dietterich (2019) ImageNet-C, and evaluated a variety of object detection models performance on corrupted images. The models suffered significant performance loss, 30~60% of the baseline performance. Nevertheless, they have proposed a mitigation method via a data augmentation technique, namely stylizing the training images. This method led to strong robustness improvements.

Kamann & Rother (2020) were the first to present a comprehensive semantic segmentation robustness study. Utilizing near 400,000 images from different datasets(Cityscapes dataset, PASCAL VOC 2012, and ADE20K), they applied image corruption types present in ImageNet-C as well as their own proposed noise model that “incorporates commonly observable behavior of cameras”. The evaluation against a range of semantic segmentation models provided the following conclusions:

- Surprisingly, the models were performing well with real-world image corruptions, with the majority of models generalizing well with image noise and image blur corruption types
- They discovered that some architectural properties played an important role in how robust the model was.
- However, they have found that image corruptions regarding the texture, such as image noise or JPEG compression, had a clear negative effect on how the models performed.

Although Kamann & Rother (2020) showed that Segmentation models are more robust to corruption than Classification and detection, there is a consensus across the three papers that digital corruption such as JPEG compression negatively affected the performance of the models.

Effects of Compression on CV models

Poyser et al. (2020) investigated the effect lossy image compression (JPEG and H.264) had on deep CNNs and the nature of its relationship. By evaluating 5 different CV tasks and models (segmentation with SegNet, human pose estimation with OpenPose, object recognition with R-CNN, human action recognition with dual-stream, and depth estimation with GAN), they have observed that there is a non-linear relationship between compression level and models' performance. There is a drastic deterioration of performance under 15% quality for the JPEG compression, and 40 for the H.264. One of the methods that improved the performance was to retrain the architectures on compressed images, which "recovered the performance to a certain degree". The conclusion stated "By using retrained models, compression can safely reach as high as 85% across all domains. In doing so, current storage costs can be markedly diminished before performance is noticeably impacted."

A year later, Ehrlich et al. (2021) attempted a quantification of the impacts and effects of JPEG compression on a range of CV tasks (Classification, Object Detection, Instance and Semantic Segmentation), while also proposing mitigation methods to reduce performance penalties. The results showed a steep performance penalty from severe to moderate JPEG compression, in which all models' performances displayed a loss in performance.

Similar to Poyser, Reich et al. (2024) attempted to also examine how JPEG and H.264 affected the predictive prowess with 4 CV tasks (image classification, object detection, optical flow estimation, and semantic segmentation). Their experiments displayed a general trend of accuracy deterioration with standard coding, with all the models suffering significantly with the compressions at inference time. In the case of maximum compression, it was shown how semantic segmentation predicts with a staggering mIoU below 10% relative to the baseline, and the same for object detection with an mean Average Precision (mAP) below 5% relative to the baseline.

Finally, Janeiro et al. (2023) added to the previous studies by analyzing and comparing image compression of traditional codecs and neural compression approaches on 3 CV tasks (image classification, object detection, and semantic segmentation). The evaluation displayed that both types of codecs produced similar results across all CV tasks; strong compressions had drastic negative effects they led to a decrease in the predictive accuracy and overall performance of the models, and However, They took an extra step and inquired about a specific detail regarding robustness and the performance of models under image compression: whether the “performance drop can be ascribed to a loss of relevant information in the compressed image, or to a lack of generalization of visual recognition models to images with compression artifacts”. By fine-tuning the models, the majority of the performance was recovered, indicating that the performance drop and recovery can be attributed to the trend across the models of not being able to generalize well to images with compression artifacts.

Practical Application of Image Compression

As an exploration of a use case solution using image compression, Faykus et al. (2022) investigated how JPEG compressions can be utilized to reduce latency of local transfer of images complemented by showing how compressed lossy images impacted semantic segmentation model (SwiftNet). As a result of their experiment, an improvement in image transfer time was observed, while the performance of the semantic segmentation model was high but witnessed steady reduction over time. One mitigation strategy to minimize this reduction was to train the model on the corresponding compression level, which caused a higher retention of mIoU levels and model performance.

Shortcomings and Contributions

The shortcomings of the previous papers are that the YOLO segmentation family was not included in any evaluation or benchmark against any Image compression, even though it gained

popularity in the past few years. Additionally, HEIC compression impacts were not evaluated next to JPEG and WebP, nor were the previous two compared in the same task.

The contribution of this paper is providing the first comparative evaluation of HEIC, JPEG, WebP compression impacts on the YOLO11 (nano) Instance Segmentation model, focusing on the trade-off between storage efficiency and models' performance.

Table 1: Literature Summary

Writers	Year	Objective	CV task and models	Key findings
Hendrycks & Dietterich	2019	To present an Image Classification robustness benchmark (ImageNet-C)	Image Classification (AlexNet, VGG family, ResNet family)	- Architectural advancements have not resulted in any major change in the relative corruption robustness - introduced several methods to increase robustness
Michaelis et al.	2020	To present an Object Detection and Instance Segmentation robustness benchmarks(Pascal-C, Coco-C and Cityscapes-C)	Object Detection and Instance Segmentation (RCNN and ResNet family)	- models suffer severe performance impairments on corrupted images - demonstrated how adding a stylized copy of the training data leads to strong robustness improvements.
Kamann & Rother	2020	To benchmark robustness of	Semantic Segmentation	Models generalize well

Writers	Year	Objective	CV task and models	Key findings
		various semantic segmentation models against image corruption	(DeepLabv3 + different architecture)	for image noise blur, but for digitally corrupted data(JPEG compression). Architectural properties are pivotal in the robustness of the models
Poyser et al.	2020	To investigate image and video compression effects on models performance.	Semantic Segmentation ,Depth estimation ,Object Detection , Human Pose estimation,Human Action Recognition (Segnet, Faster-RCNN, and various CNN models)	- performance decreases significantly below a JPEG quality level of 15% - Retraining on precompressed imagery recovers performance by up to 78.4% in some cases. - Current storage costs can be markedly diminished

Writers	Year	Objective	CV task and models	Key findings
				before performance is impacted.
Ehrlich et al.	2021	To study effects of JPEG compression on CV tasks and mitigation strategies	Classification, Detection, Instance Segmentation , Semantic Segmentation (ResNet and RCNN families)	• JPEG compression has a steep penalty for heavy to moderate compression settings. • Proposed mitigation strategy achieved higher results than other mitigation methods
Faykus et al.	2022	To examine JPEG compression effects on semantic segmentation accuracy and reduction of image transfer time	Semantic Segmentation (SwiftNet)	• The mIoU on compressed images remains high but steadily decreases • It can be mitigated by training SwiftNet on the corresponding

Writers	Year	Objective	CV task and models	Key findings
				compression levels to retain higher levels of mIoU.
Janeiro et al.	2023	To study Impact of traditional and state of art image codecs on models' performance	image classification , object detection , semantic segmentation (ResNet-50 and Swin-T as backbones)	• Image compression leads to a degradation of visual recognition performance • Performance reduction can be attributed to the models' inability to generalize to images with compression artefacts, rather than the presence of compression artefacts increasing the difficulty of recognition.
Reich et al.	2024	To examine impact of JPEG	Semantic Segmentation,	The accuracy of the predictions

Writers	Year	Objective	CV task and models	Key findings
		compression on models	Object Detection (ResNet family as backbone)	were severely deteriorated with the standard coding (JPEG).

METHODOLOGY

Research Method

This study is a quantitative experimental benchmark aimed to evaluate the robustness of an instance segmentation model under different image formats compressions. Training conditions are fixed, and images formats at inference are varied to measure performance caused by distribution shifts caused by compression.

Problem Definition

The study investigates the robustness of YOLO11n instance segmentation model when evaluated on different format compressions. The objective is to analyze the trade-off between segmentation accuracy and storage efficiency by comparing model performance across different image formats and compression levels.

Dataset and Data Collection

The Cityscapes dataset - which is recognized for semantic and instance segmentation tasks focused on urban street scenes - is used with its official train, validation, and test splits. The model is trained on PNG images from the training set, and an evaluation of PNG images is carried out and considered a baseline. Afterwards, test set images are converted from PNG to JPEG, WebP, and HEIC formats at different compression levels using Python scripts based on the Pillow library, and the ground-truth annotations remain the same across all formats and compressions.

Methods, Techniques, and Tools Used

The model selected is the YOLO11 architecture, specifically the “Nano” version (YOLO11n-seg), due to its efficiency and suitability for edge devices. YOLO was first proposed by Redmon et al. (2016). It was presented as a unification of separate components of object detection into a single network, hence the model being a single shot detector. It deviates from earlier methodologies where two-stage approaches proposed regions of interest first and then classification. By treating object detection as a single regression problem, YOLO network processes the entire image simultaneously, directly predicting bounding box coordinates and corresponding class probabilities as shown in Figure 1 (Redmon et al., 2016).

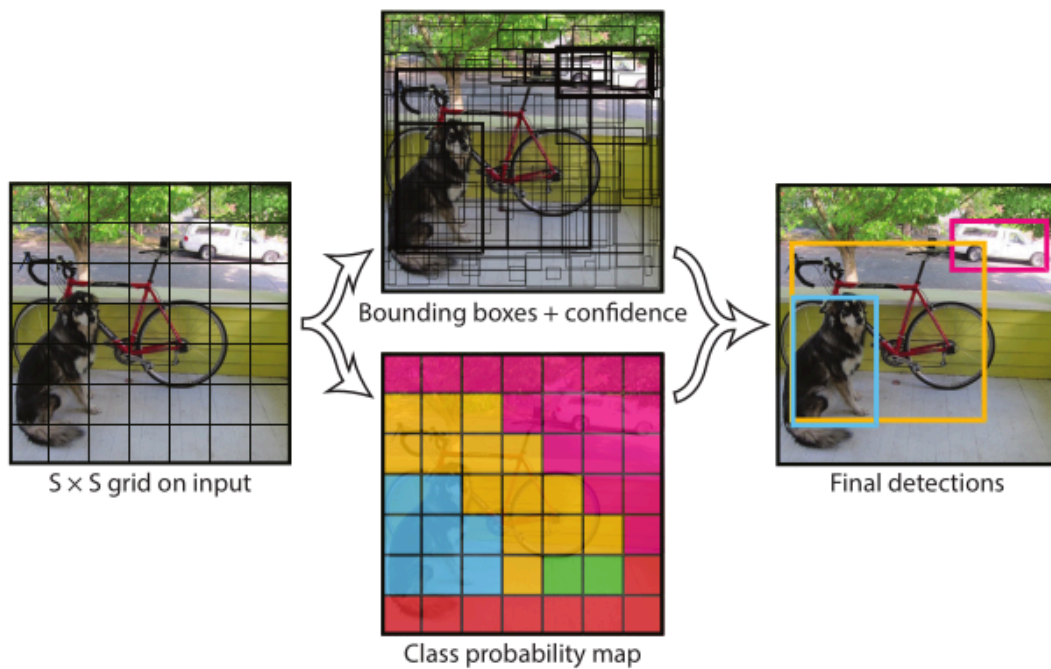


Figure 1: Visualization of how YOLO detection works

For data manipulation, the Pillow library in Python was the primary tool used to convert PNG images into different formats and to apply the seven distinct compression levels for each format. Data pre-processing, model training, and inferencing was implemented using Python and Python libraries such as Pandas, Numpy, and Ultralytics.

Finally, A score was crafted to measure the storage performance trade-off, called the Compression Score.

$$\begin{aligned} \text{CS} &= \left(\frac{\overline{\text{FileSize}}_{\text{PNG}} - \overline{\text{FileSize}}_{\text{compression}}}{\overline{\text{FileSize}}_{\text{PNG}}} * 100 \right) - \left(\frac{\text{mAP}_{\text{PNG}} - \text{mAP}_{\text{compression}}}{\text{mAP}_{\text{PNG}}} * 100 \right) \\ &= \text{SizeReduction} - \text{mAPDrop} \end{aligned}$$

where the reduction of the original file and of the performance are considered as percentages and the difference is taken. This score is an indicator of the trade-off and the relationship between storage and performance, where the larger score indicates a more optimal and preferable setting. It should be mentioned that this score is not to be taken as an absolute measure, rather it is comprehended in the presence of other metrics as well.

Additionally, three further metrics were derived to analyze the trade-off from different perspectives.

The Efficiency Ratio measures how many units of size reduction are achieved per unit of mAP drop, defined as:

$$\text{ER} = \frac{\text{SizeReduction}}{\text{mAPDrop}}$$

where a higher ratio indicates greater storage savings relative to performance loss.

The Weighted Compression Score (WCS) is an extension of the Compression Score that introduces a weighting factor (lambda) to penalize mAP drop more heavily depending on the use case priority. The formula is defined as:

$$\text{WCS} = \text{SizeReduction} - \lambda * \text{mAPDrop}$$

where lambda is a multiplier that reflects the relative importance of accuracy versus storage. A higher lambda places more emphasis on preserving model performance, while a lower lambda prioritizes storage savings. At lambda = 1, the WCS reduces to the original Compression Score. This metric is particularly useful for practitioners who need to make format and compression decisions based on their specific operational constraints.

Marginal Efficiency measures the incremental efficiency of each step down in quality level, calculated as the ratio of additional size reduction gained to the additional mAP drop incurred when moving from one quality level to the next lower level. The formula is:

$$ME = \frac{\text{SizeReduction}_{q_n} - \text{SizeReduction}_{q_{n+1}}}{\text{mAPDrop}_{q_n} - \text{mAPDrop}_{q_{n+1}}}$$

To generalize the threshold for different deployment priorities, the same lambda parameter is applied here. A quality step is considered efficient if $ME > \lambda$. At $\lambda = 1$ (the baseline), storage and accuracy are treated as equally important in a 1:1 ratio: 1% of storage savings justifies 1% of mAP loss. At $\lambda > 1$, accuracy is prioritized, requiring greater storage savings to justify each unit of accuracy loss; for example, $\lambda = 5$ represents a 5:1 ratio where 5% storage savings are needed to justify 1% mAP loss. At $\lambda < 1$, storage savings are prioritized, allowing even steps with $ME < 1$ to be acceptable if $ME > \lambda$.

Analysis Plan and Implementation Steps

YOLO11n-seg model was trained from scratch on the uncompressed Cityscapes training data images. The model was trained for 150 epochs, with 2048 as the image size. Following this, a baseline evaluation was conducted on the original PNG test split. Afterwards, the test split images were converted into JPEG, WebP, and HEIC formats. For each format, seven versions of the dataset were generated, corresponding to different quality values which are {90, 75, 50, 30, 15, 10, 5}. Although they look the same, each format treats quantization differently than other formats at the same quality level, So a JPEG of quality 5 does not equate WebP of quality 5. Finally, the model was evaluated on each of these compressed datasets, metrics and scores were recorded. Instance Segmentation performance is evaluated via the mean Average Precision(mAP), specifically the mask mAP@50, mask mAP@50–95, and box @50, while storage efficiency is evaluated via the average file size measured in kilobytes. These metrics are later used to calculate the Compression Score, which acts as an indicator to the trade-off and the relationship. Since YOLO cannot evaluate HEIC files using the validation method, the

HEIC files were converted to PNG files, taking into consideration that PNG files are lossless, and that they can act as containers that would preserve the image's original distribution without any shift. To ensure that, Structural similarity index measure (SSIM) (Wikipedia contributors, 2025) was carried out between the prior HEIC and the new PNG containing it, and a score above 99% positively indicated that there has not been any distributional shifts in terms of digital corruption. The flowchart shown in Figure 2 shows these steps.

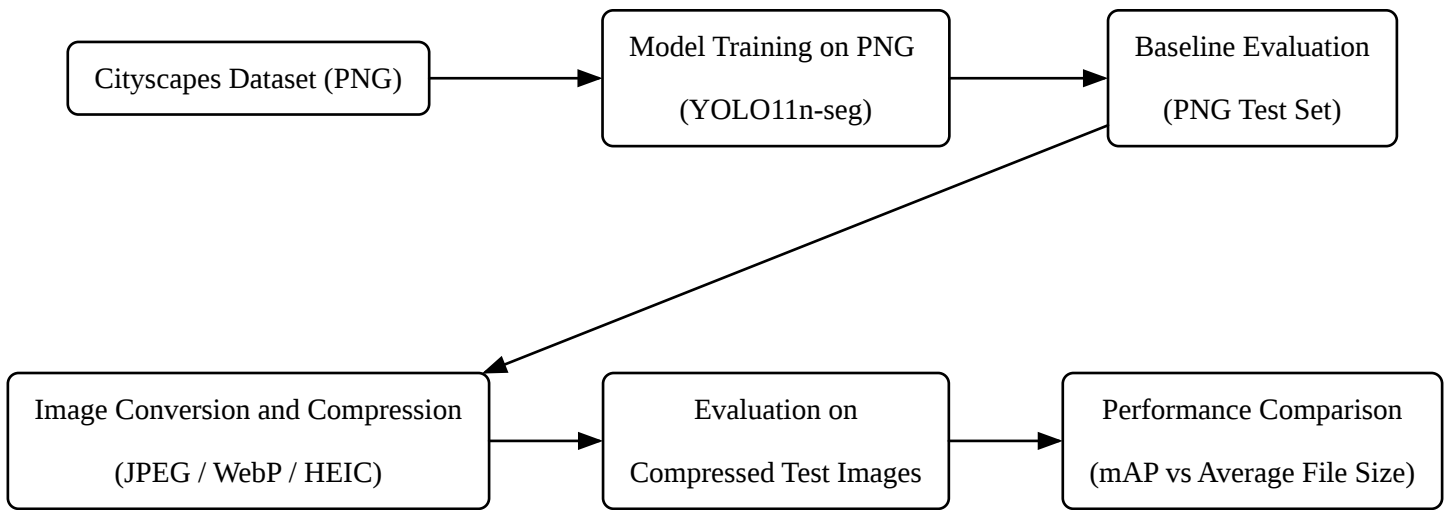


Figure 2: Implementation steps

Assumptions and Limitations

It is assumed that image compression represents realistic deployment distortions and that annotations remain valid across image formats. The study is limited to a single dataset, a single model scale (YOLO11n), and three image formats. Another limitation is that YOLO cannot use the `model.val` function with the HEIC format, hence it was converted to PNG to continue the validation process. Although the SSIM (structural similarity index measure) between the HEIC image and the PNG was above 99% indicating that both images are similar and that no distribution shift has occurred in terms of digital corruption, it still counts as a limitation. Additionally, the Marginal Efficiency metric's cross-format comparison is limited by the fact that quality parameters are not standardized across encoders - a quality value of 75 in JPEG does not represent the same compression level as quality 75 in HEIC or WebP, meaning step-by-step transitions are not directly comparable across formats. The metric is therefore most reliable when used for within-format analysis.

RESULTS AND ANALYSIS

The baseline that the compression results are compared against is the evaluation of PNG images on the model trained on PNG images. The Mask mAP@50 95 for the baseline is 0.366, which will be used in the calculation of how much the mAP dropped.

Performance by File Size

The individual bar charts for each format show the relationship between average file size and Mask mAP@50 95.

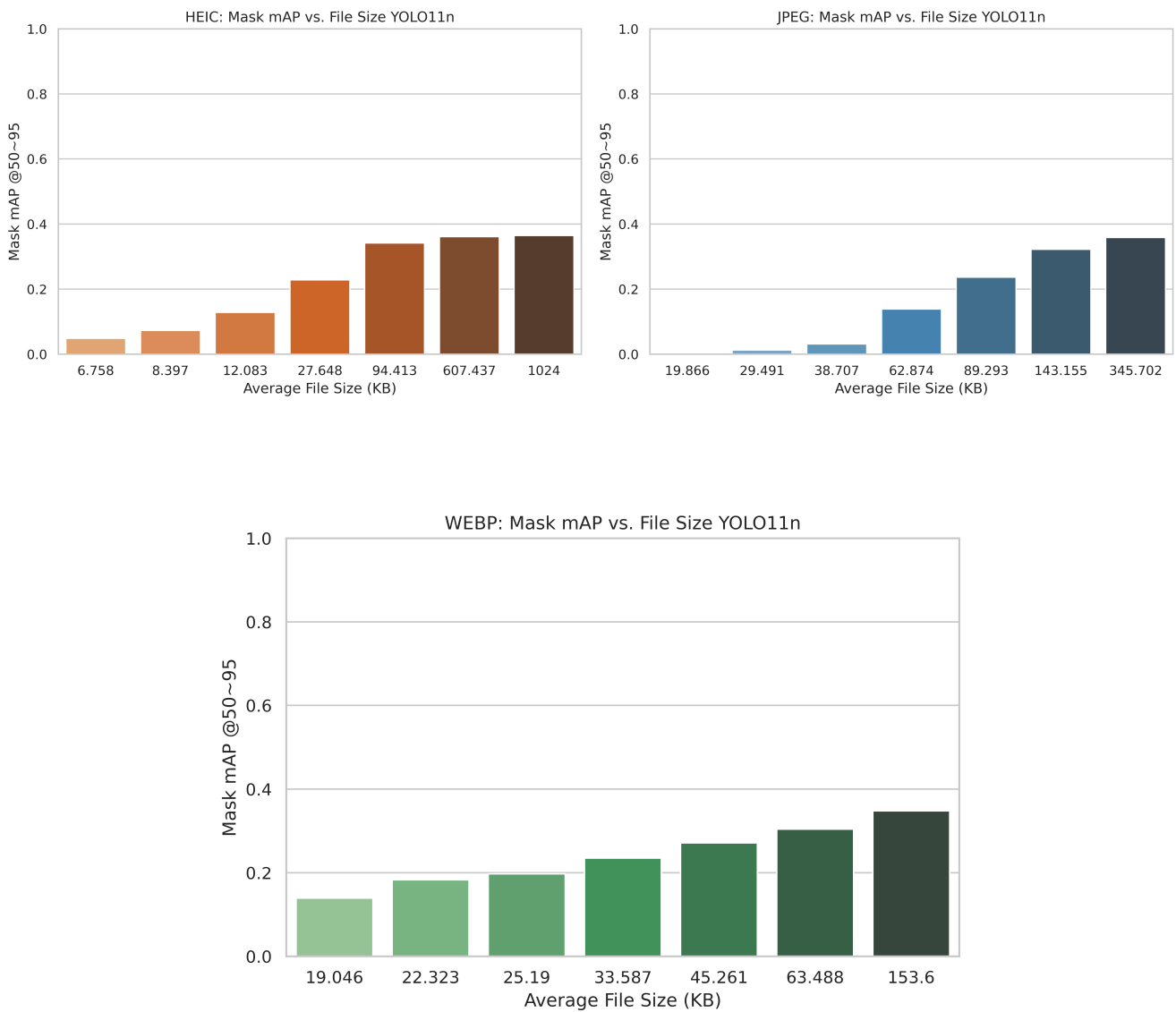


Figure 3: Bar chart of Average file size of each format and its respective performance (Mask mAP@50 95)

HEIC achieves higher mAP at smaller file sizes compared to JPEG and WEBP. At approximately 94 KB, HEIC reaches an mAP of 0.34, while JPEG requires approximately 345 KB to reach a similar mAP of 0.35. WEBP requires approximately 153 KB to reach an mAP of 0.34. The combined bar charts show how the formats interleave across the size-performance spectrum.

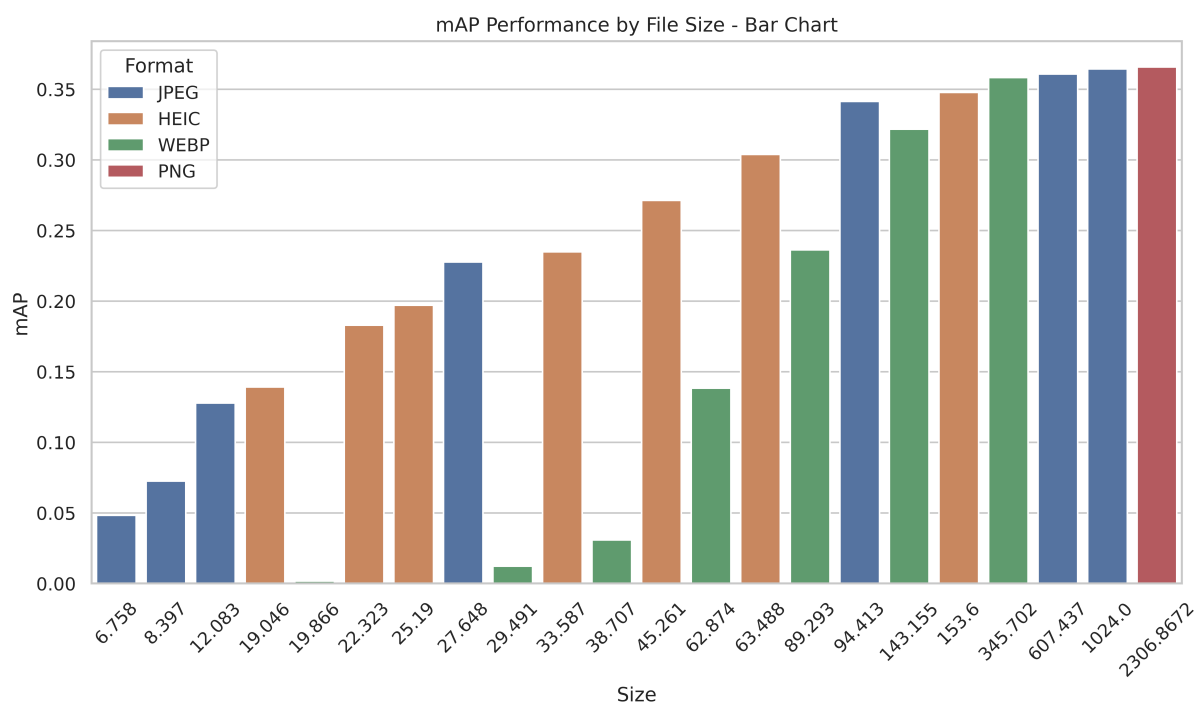


Figure 4: mAP Performance by File Size - Bar Chart (Ordered by size)

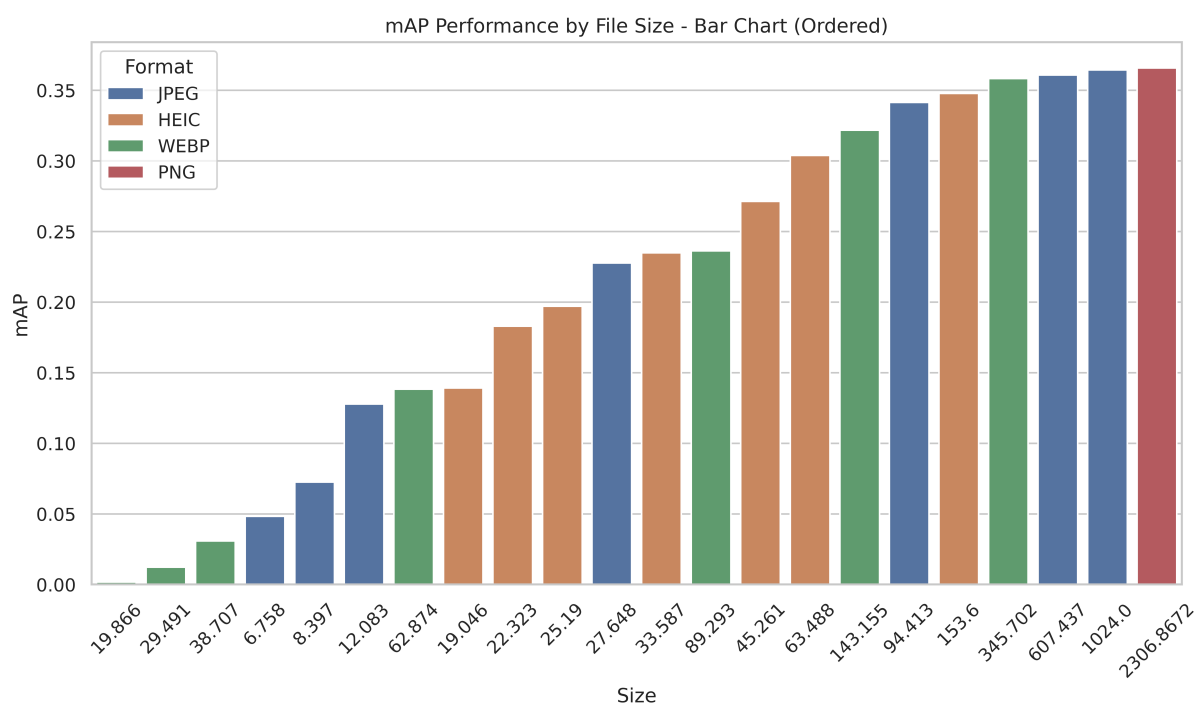


Figure 5: mAP Performance by File Size - Bar Chart (Ordered by mAP)

At the lower end of the size spectrum (below 50 KB), WEBP and JPEG show lower mAP values than HEIC. HEIC maintains a performance advantage in the 20-100 KB range. At higher file sizes (above 150 KB), all three formats converge toward similar mAP values, with JPEG and HEIC reaching approximately 0.35-0.36.

The line graph shows the mAP performance curve for each format as file size increases.

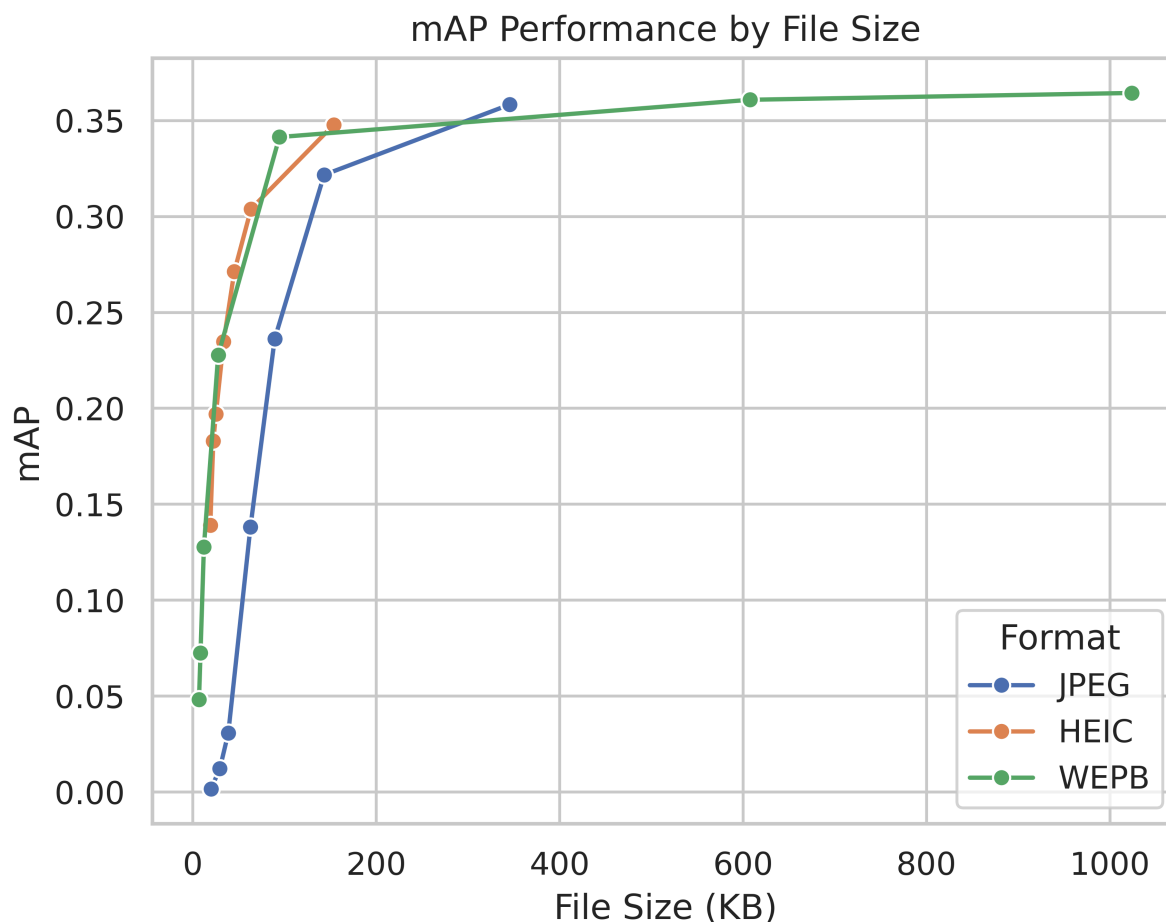


Figure 6: mAP Performance by File Size - Line Graph

HEIC shows the steepest initial rise in mAP, reaching 0.34 mAP at approximately 94 KB. WEBP follows a more gradual curve, reaching 0.34 mAP at approximately 153 KB. JPEG shows the slowest initial rise, requiring approximately 345 KB to reach 0.35 mAP. All three formats plateau near 0.35-0.36 mAP at higher file sizes.

Efficiency Ratio Analysis

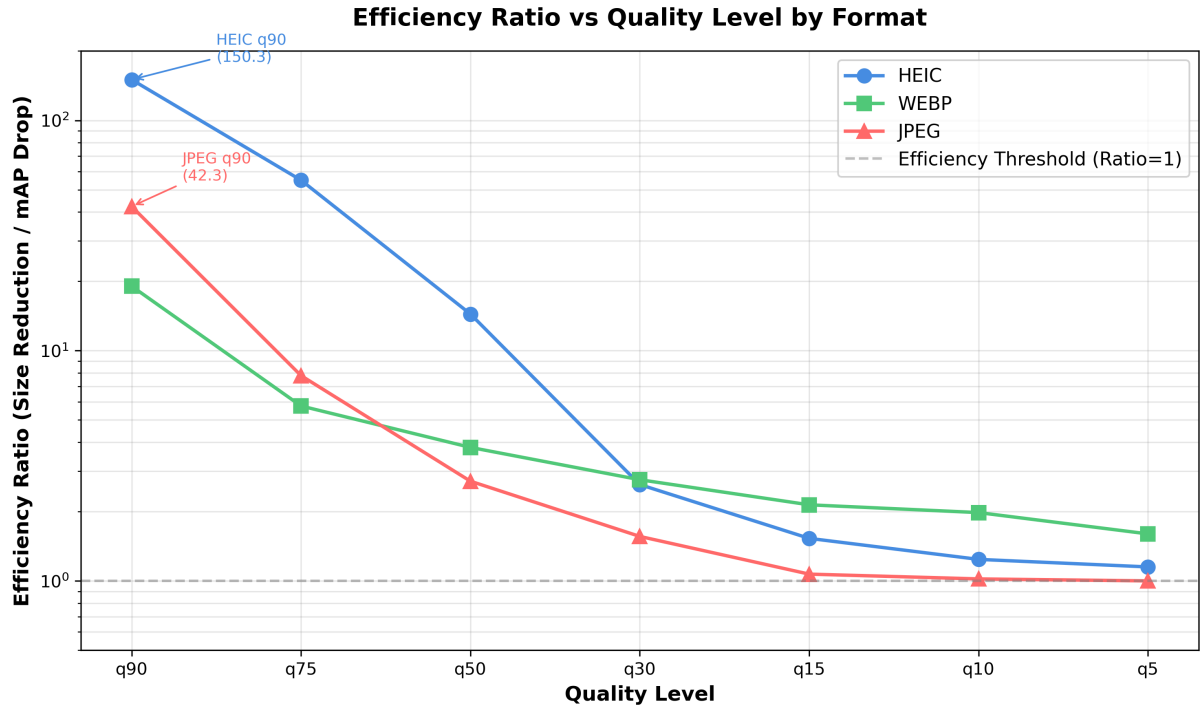


Figure 7: Efficiency Ratio vs Quality Level by Format

HEIC q90 shows the highest efficiency ratio of 150.30, indicating extreme efficiency at high quality.

JPEG q90 follows with 42.29, and WEBP q90 has 19.05.

The graph reveals a non-linear, exponential decay relationship for all formats. HEIC maintains an efficiency ratio above 10 until q50, then drops sharply. JPEG drops below 10 by q75. WEBP shows the most gradual and stable decline across all quality levels.

All three formats approach an efficiency ratio of 1.0 at the lowest quality levels (q5-q10), indicating diminishing returns where additional compression yields minimal size savings for significant performance loss.

Weighted Compression Score

Three lambda values were evaluated to represent different deployment scenarios. The three lambda values selected - 0.5, 2, and 5 - represent storage-priority, balanced, and accuracy-priority deployment scenarios respectively, and are chosen to span a range of practical use cases rather than to represent specific application requirements.

Table 2: Top configurations ranked by Weighted Compression Score at lambda=0.5 (Storage Priority)

Format	Quality	Size Reduction (%)	WCS (lambda=0.5)
HEIC	q50	95.91	92.58
WEBP	q90	93.34	90.89
JPEG	q75	93.79	87.77

Format	Quality	Size Reduction (%)	WCS (lambda=0.5)
WEBP	q75	97.25	88.80
HEIC	q75	73.67	73.00
JPEG	q90	85.01	84.00
WEBP	q50	98.04	85.14
HEIC	q90	55.61	55.42
JPEG	q50	96.13	78.42

Table 3: Top configurations ranked by Weighted Compression Score at lambda=2 (Balanced Priority)

Format	Quality	Size Reduction (%)	WCS (lambda=2)
WEBP	q90	93.34	83.54
HEIC	q50	95.91	82.61
JPEG	q90	85.01	80.99
HEIC	q75	73.67	70.99
JPEG	q75	93.79	69.71
WEBP	q75	97.25	63.45
HEIC	q90	55.61	54.87
WEBP	q50	98.04	46.42
JPEG	q50	96.13	25.31

Table 4: Top configurations ranked by Weighted Compression Score at lambda=5 (Accuracy Priority)

Format	Quality	Size Reduction (%)	WCS (lambda=5)
JPEG	q90	85.01	74.96
WEBP	q90	93.34	68.84
HEIC	q75	73.67	66.97
HEIC	q50	95.91	62.66
JPEG	q75	93.79	33.59
WEBP	q75	97.25	12.75
HEIC	q90	55.61	53.76
JPEG	q50	96.13	-80.92
WEBP	q50	98.04	-31.01

HEIC configurations appear in higher positions at lower lambda values and shift to lower positions as lambda increases, while JPEG q90 rises to the top position at lambda=5.

Marginal Efficiency

Table 5: Marginal Efficiency across quality transitions for HEIC (top), JPEG (middle), and WEBP (bottom)

Quality Transition	Add. Size Red. (%)	Add. mAP Drop (%)	ME	Eff. ($\lambda=0.5$)	Eff. ($\lambda=1$)	Eff. ($\lambda=2$)
q90 to q75	18.06	0.97	18.62	✓	✓	✓
q75 to q50	22.24	5.31	4.19	✓	✓	✓
q50 to q30	2.89	31.09	0.09	✗	✗	✗
q30 to q15	0.68	27.33	0.02	✗	✗	✗
q15 to q10	0.16	15.10	0.01	✗	✗	✗
q10 to q5	0.07	6.64	0.01	✗	✗	✗

Efficient at $\lambda = 5$: q90→q75 only.

Quality Transition	Add. Size Red. (%)	Add. mAP Drop (%)	ME	Eff. ($\lambda=0.5$)	Eff. ($\lambda=1$)	Eff. ($\lambda=2$)
q90 to q75	8.78	10.03	0.88	✓	✗	✗
q75 to q50	2.34	23.37	0.10	✗	✗	✗
q50 to q30	1.14	26.78	0.04	✗	✗	✗
q30 to q15	1.05	29.37	0.04	✗	✗	✗
q15 to q10	0.40	5.09	0.08	✗	✗	✗
q10 to q5	0.42	2.87	0.15	✗	✗	✗

Efficient at $\lambda = 5$: None.

Quality Transition	Add. Size Red. (%)	Add. mAP Drop (%)	ME	Eff. ($\lambda=0.5$)	Eff. ($\lambda=1$)	Eff. ($\lambda=2$)
q90 to q75	3.91	12.00	0.33	✗	✗	✗
q75 to q50	0.79	8.91	0.09	✗	✗	✗
q50 to q30	0.50	9.99	0.05	✗	✗	✗
q30 to q15	0.37	10.32	0.04	✗	✗	✗
q15 to q10	0.12	3.86	0.03	✗	✗	✗
q10 to q5	0.14	11.96	0.01	✗	✗	✗

Efficient at $\lambda = 5$: None.

At $\lambda = 1$, HEIC has marginal efficiency values above the threshold for the q90 to q75 and q75 to q50 transitions, while JPEG and WEBP have values below the threshold across all transitions.

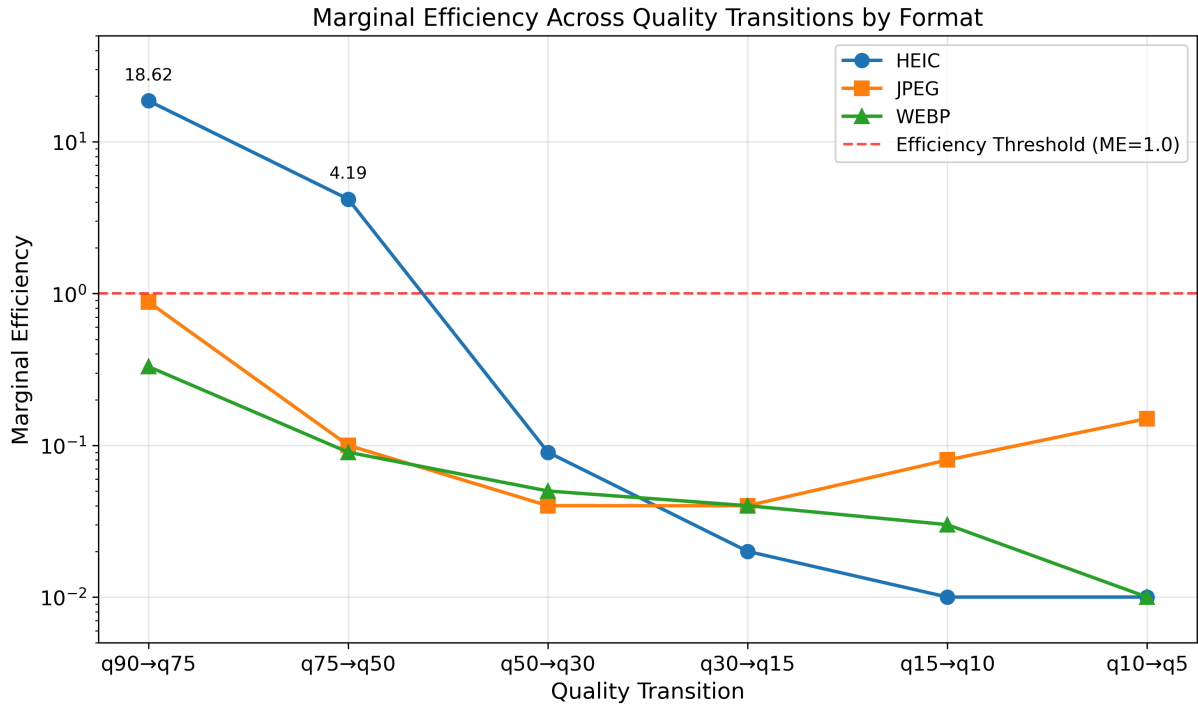


Figure 8: Marginal Efficiency across quality transitions by format. The dashed threshold line represents $ME = \lambda$, with $\lambda = 1$ as the default shown.

Threshold-Based Observations

The following table shows the format and quality configuration that achieves the highest size reduction for various minimum mAP requirements.

Table 6: Configurations achieving highest size reduction at different minimum mAP thresholds

Minimum mAP	Optimal Format	Quality	Size Reduction (%)	mAP Drop (%)
0.36	HEIC	q75	73.67	1.34
0.35	JPEG	q90	85.01	2.01
0.34	HEIC	q50	95.91	6.65
0.30	WEBP	q75	97.25	16.90
0.25	WEBP	q50	98.04	25.81

CONCLUSION

This study benchmarks the performance of the YOLO11n instance segmentation model under three lossy image compression formats - JPEG, HEIC, and WebP - across seven quality levels, with the aim of measuring how storage efficiency and model performance trade off against each other. The model was trained on PNG images and evaluated on compressed test sets, with PNG serving as the baseline.

The results show that HEIC and WebP provide a more favorable storage-performance balance than JPEG across most quality settings. HEIC in particular demonstrated the highest storage efficiency, reaching comparable mAP values at significantly smaller file sizes. JPEG, while competitive at high quality settings, was the weakest format under aggressive compression, and the only format to produce a negative Compression Score at extreme quality levels.

The metric framework introduced in this study, which includes the Compression Score, Weighted Compression Score, Efficiency Ratio, and Marginal Efficiency, provides different perspectives on the same tradeoff. Together they show that HEIC is the only format with multiple efficient compression operating points before a sharp performance cliff at q30. JPEG and WebP each have at most one efficient operating point at q90, beyond which compression steps are not justified regardless of deployment priority.

The study is limited to a single model, a single dataset, and three formats. The Marginal Efficiency metric's cross-format comparison is also limited by the fact that quality parameters are not standardized across encoders, meaning the same quality value does not represent the same compression level in different formats.

Future work could extend the benchmark to other YOLO scales, additional CV tasks, and formats such as JPEG 2000 or AVIF, as well as exploring strategies to improve model robustness under compression.

BIBLIOGRAPHY

- Bojarski, M., Testa, D. D., Dworakowski, D., Firner, B., Flepp, B., Goyal, P., Jackel, L. D., Monfort, M., Muller, U., Zhang, J., Zhang, X., Zhao, J., & Zieba, K. (2016,). End to End Learning for Self-Driving Cars. <https://arxiv.org/abs/1604.07316>
- Cordts, M., Omran, M., Ramos, S., Rehfeld, T., Enzweiler, M., Benenson, R., Franke, U., Roth, S., & Schiele, B. (2016,). The Cityscapes Dataset for Semantic Urban Scene Understanding. <https://arxiv.org/abs/1604.01685>
- Ehrlich, M., Davis, L., Lim, S.-N., & Shrivastava, A. (2021,). Analyzing and Mitigating JPEG Compression Defects in Deep Learning. <https://arxiv.org/abs/2011.08932>
- Esteva, A., Kuprel, B., Novoa, R. A., Ko, J. M., Swetter, S. M., Blau, H. M., & Thrun, S. (2017). Dermatologist-level classification of skin cancer with deep neural networks. *Nature*, 542, 115–118. <https://api.semanticscholar.org/CorpusID:3767412>
- Faykus, M. H., Selee, B., Calhoun, J. C., & Smith, M. C. (2022). Lossy Compression to Reduce Latency of Local Image Transfer for Autonomous Off-Road Perception Systems. 2022 IEEE International Conference on Big Data (Big Data), , 3146–3152. <https://doi.org/10.1109/BigData55660.2022.10020267>
- Foxon, T. (2013). Technological Lock-In (pp. 123–127). <https://doi.org/10.1016/B978-0-12-375067-9.00067-X>
- Girshick, R., Donahue, J., Darrell, T., & Malik, J. (2014,). Rich feature hierarchies for accurate object detection and semantic segmentation. <https://arxiv.org/abs/1311.2524>
- He, K., Gkioxari, G., Dollár, P., & Girshick, R. (2018,). Mask R-CNN. <https://arxiv.org/abs/1703.06870>
- Hendrycks, D., & Dietterich, T. (2019,). Benchmarking Neural Network Robustness to Common Corruptions and Perturbations. <https://arxiv.org/abs/1903.12261>
- Jalilian, E., Hofbauer, H., & Uhl, A. (2022). Iris Image Compression Using Deep Convolutional Neural Networks. *Sensors*, 22(7). <https://www.mdpi.com/1424-8220/22/7/2698>
- Janeiro, J. M., Frolov, S., El-Nouby, A., & Verbeek, J. (2023,). Are Visual Recognition Models Robust to Image Compression?. <https://arxiv.org/abs/2304.04518>
- Kamann, C., & Rother, C. (2020). Benchmarking the Robustness of Semantic Segmentation Models. 2020 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), 8825–8835. <https://doi.org/10.1109/cvpr42600.2020.00885>
- Krizhevsky, A., Sutskever, I., & Hinton, G. E. (2012). Imagenet classification with deep convolutional neural networks. *Advances in Neural Information Processing Systems*, 25.
- Michaelis, C., Mitzkus, B., Geirhos, R., Rusak, E., Bringmann, O., Ecker, A. S., Bethge, M., & Brendel, W. (2020,). Benchmarking Robustness in Object Detection: Autonomous Driving when Winter is Coming. <https://arxiv.org/abs/1907.07484>

NVIDIA, N. D. (2026,). NVJPEG. <https://developer.nvidia.com/nvjpeg>

Poyser, M., Atapour-Abarghouei, A., & Breckon, T. P. (2020,). On the Impact of Lossy Image and Video Compression on the Performance of Deep Convolutional Neural Network Architectures. <https://arxiv.org/abs/2007.14314>

Redmon, J., Divvala, S., Girshick, R., & Farhadi, A. (2016,). You Only Look Once: Unified, Real-Time Object Detection. <https://arxiv.org/abs/1506.02640>

Reich, C., Hahn, O., Cremers, D., Roth, S., & Debnath, B. (2024). A Perspective on Deep Vision Performance with Standard Image and Video Codecs. 2024 IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops (CVPRW), 5712–5721. <https://doi.org/10.1109/cvprw63382.2024.00580>

Ren, S., He, K., Girshick, R., & Sun, J. (2016,). Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks. <https://arxiv.org/abs/1506.01497>

Schroff, F., Kalenichenko, D., & Philbin, J. (2015). FaceNet: A unified embedding for face recognition and clustering. 2015 IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 815–823. <https://doi.org/10.1109/cvpr.2015.7298682>

Ultralytics. (2026,). Home. <https://docs.ultralytics.com/>

Wang, S., Veldhuis, R., Brune, C., & Strisciuglio, N. (2024,). A Survey on the Robustness of Computer Vision Models against Common Corruptions. <https://arxiv.org/abs/2305.06024>

Wikipedia contributors. (2025,). Structural similarity index measure — Wikipedia, The Free Encyclopedia. https://en.wikipedia.org/w/index.php?title=Structural_similarity_index_measure&oldid=1320620758

Wikipedia contributors. (2026,). JPEG 2000 — Wikipedia, The Free Encyclopedia. https://en.wikipedia.org/w/index.php?title=JPEG_2000&oldid=1334323191